

Academic Question Answering System Using RAG

Retrieval-Augmented Generation with PDF Page Citations

ABSTRACT

The Academic Question Answering System applies Retrieval-Augmented Generation (RAG) to answer questions using a selected collection of academic PDF documents. It extracts text while preserving page information, divides the content into overlapping chunks, creates semantic embeddings, and stores them in a vector database. When a user asks a question, the system retrieves relevant passages and supplies them to a language model to produce an answer grounded in the source material.

Source and page citations make it easier to verify responses against the original PDFs. The project therefore focuses on two connected goals: retrieving useful evidence and presenting answers whose supporting passages can be checked. Testing evaluates retrieval behavior, answer faithfulness, and citation correctness.

PROJECT WORKFLOW

PDF documents → Text extraction → Chunking → Embeddings → Vector search → Grounded answer with citations

1. Technology Stack

The system is organized as a Python-based retrieval and generation pipeline. The components below reflect the tools used or established in the current project workflow; optional components are identified separately.

Python

Core language for document processing, retrieval, answer generation, and evaluation.

PyMuPDF

Extracts selectable text from PDFs and helps preserve page-level source information.

LangChain text splitter

Splits extracted text into smaller overlapping chunks while retaining useful context.

Sentence Transformers — BAAI/bge-small-en-v1.5

Creates semantic embeddings for document chunks and user questions.

ChromaDB

Stores vectors, text chunks, and metadata for similarity-based retrieval.

Google Gemini API

Generates responses from the passages retrieved for a question.

python-dotenv

Loads configuration and API credentials from environment variables.

Supporting configuration

- Chunking baseline: approximately 250 tokens per chunk with 60 tokens of overlap; tune this using evaluation results.
- Metadata should preserve the source filename, original page number, and a stable chunk identifier.
- LangChain orchestration or Streamlit interface components should be listed as implemented only if they are present in the actual repository.

Keep API keys out of source files and version control. Record exact installed package versions and model identifiers in the project environment for reproducibility.

2. Flow and Architecture

The architecture has two workflows: indexing documents before questions are asked, and retrieving evidence whenever a question is submitted.

A. Document indexing

PDF collection → Text extraction → Text cleaning → Chunking → Source/page metadata → Embeddings → ChromaDB

During ingestion, every chunk must remain linked to its original document and page. This relationship allows the system to identify where retrieved information came from.

B. Question answering

User question → Query embedding → Similarity search → Relevant chunks → Prompt with context → Gemini response → Answer and citations

C. Main components

- **Ingestion:** reads PDFs and captures text with page information.
- **Preprocessing:** cleans extracted text and creates overlapping chunks.
- **Embedding model:** represents passages and questions as vectors.
- **Vector database and retriever:** finds passages semantically related to the question.
- **Generation layer:** uses retrieved evidence to formulate the answer.
- **Citation layer:** displays source and page details from stored metadata.
- **Evaluation layer:** checks retrieval relevance, answer faithfulness, and citation validity.

Citation handling

Citations should be assembled from stored metadata rather than invented by the language model. When the indexed documents do not contain sufficient evidence, the system should say so instead of presenting an unsupported answer.

3. Directory Structure

The following is a clear logical layout for the project pipeline. Match the names to the actual repository before final submission; the tree is a documentation guide, not a claim that every folder already exists.

```
Academic-RAG-System/  
■■■ extract_text.py  
■■■ chunk_text.py  
■■■ answer.py  
■■■ evaluate_rag.py  
■■■ data/  
    ■■■ raw_pdfs/  
    ■■■ extracted_text/  
    ■■■ cleaned_text/  
    ■■■ processed_chunks/  
    ■■■ evaluation/  
    ■■■ chroma_db/
```

File and folder roles

- **extract_text.py** — extracts PDF text and retains source/page metadata.
- **chunk_text.py** — cleans and splits extracted content into overlapping chunks.
- **answer.py** — connects question processing, retrieval, generation, and citation output.
- **evaluate_rag.py** — runs checks against the RAG workflow.
- **data/raw_pdfs/** — original PDF files used as the knowledge source.
- **data/extracted_text/** and **data/cleaned_text/** — intermediate text outputs for inspection.
- **data/processed_chunks/** — prepared chunks and their metadata.
- **data/evaluation/** — evaluation questions and related evaluation artefacts.
- **data/chroma_db/** — persistent ChromaDB index, when persistence is enabled.

Use consistent names and keep generated data separate from source code. If the implementation combines stages into fewer files, document the real structure rather than creating empty folders just to match this example.

4. Testing and Performance

The project has previously reported successful evaluation and passed tests after changes. The following checks describe the intended coverage; report numerical scores or timing results only after measuring them on the project dataset.

Testing coverage

- **PDF extraction:** extracted text is readable and page references correspond to the source PDF.
- **Metadata integrity:** retrieved chunks retain valid source filenames and original page numbers.
- **Relevant question:** the retriever returns supporting passages and the answer reflects them.
- **Paraphrased question:** semantic retrieval finds relevant evidence despite different wording.
- **Unsupported question:** the system avoids fabricating information when evidence is absent.
- **Citation validation:** each cited page exists and supports the associated answer.
- **Error handling:** empty input, missing documents, and API failures produce clear feedback.

Performance measures

- **Retrieval:** Precision@K, Recall@K, or ranking measures on a labelled question set.
- **Answer quality:** factual correctness, completeness, and faithfulness to retrieved passages.
- **Citation quality:** validity of source/page references and support for answer claims.
- **Latency:** indexing duration and average response time across repeated questions.

5. Future Scope

- Add OCR for scanned PDFs and improve extraction of tables and equations.
- Evaluate hybrid keyword/vector retrieval and reranking to improve passage selection.
- Provide clickable citations and highlight the supporting source passages.
- Expand the document collection and create a larger, independently labelled evaluation set.
- Improve multi-document reasoning, unsupported-question handling, and conversation history.
- Add deployment, monitoring, and more robust error handling where needed.

6. Conclusion

The project combines PDF extraction, chunking, embeddings, vector search, and language-model generation to answer academic questions from supplied documents. Preserving source and page metadata supports verifiable citations, while systematic testing helps identify weaknesses in retrieval, answer faithfulness, and citation accuracy. Further improvements should be selected using measured results from a representative evaluation set.